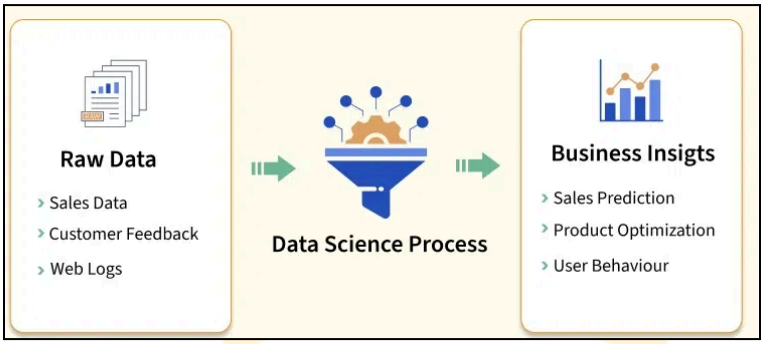
Data Science

Introduction

Data Science is the discipline of extracting meaningful insights and knowledge from raw data using statistical methods, algorithms, and computational tools. It combines **statistics**, **computer science**, and **domain knowledge** to analyze and interpret data effectively.



Python plays a central role in data science due to its simplicity, readability, and extensive ecosystem of libraries — including **NumPy**, **Pandas**, **Matplotlib**, and **Scikit-learn**.The first step in learning data science is to understand the **Python environment**, **programming constructs**, and **data manipulation libraries** that underpin analytical workflows.

Python Programming

1. Python Environment

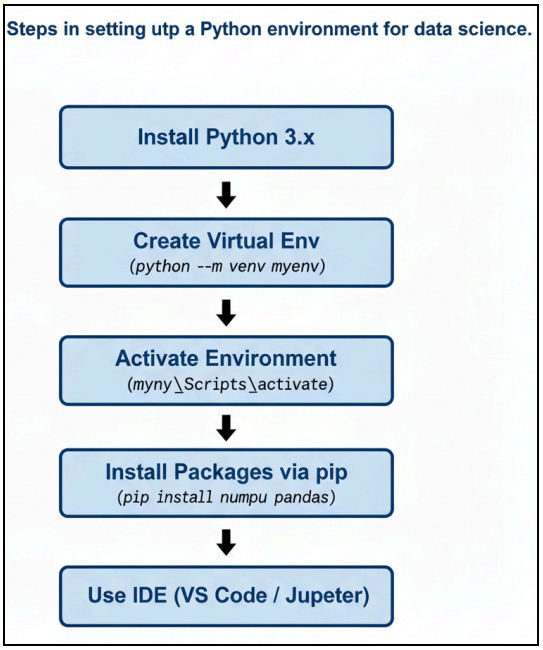
The Python programming environment refers to the configuration and setup that allows users to write, test, and execute code efficiently. A well-prepared environment ensures smooth workflow management, modularity, and reproducibility.

Python can be downloaded from the **official Python Software Foundation** website.

After installation, the command below verifies the version:

```
python --version
```

Since Python 2 is deprecated, the latest **Python 3.x** version is recommended.



1.1 Virtual Environment

Virtual environments help create isolated workspaces for different projects, preventing dependency conflicts.

They can be created using:

```
python -m venv myenv
```

Activated with:

- **Windows:** `myenv\Scripts\activate`
- **macOS/Linux:** `source myenv/bin/activate`

1.2 Package Management

Python packages extend its functionality. The **pip** tool manages installations:

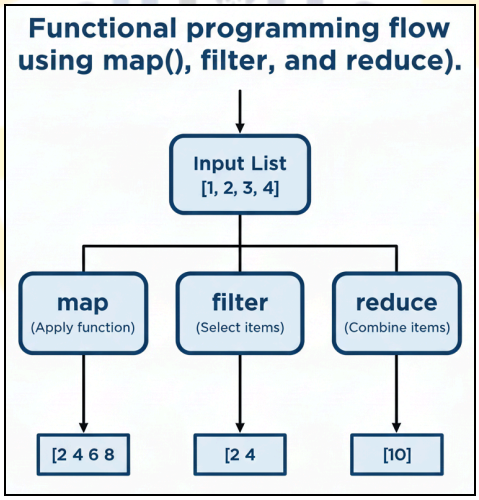
```
pip install numpy pandas matplotlib
```
For project reproducibility, dependencies are saved in:
```
pip install -r requirements.txt
```
1.3 Integrated Development Environments (IDEs)
Common IDEs for Python development include:

- **VS Code** – Lightweight and extensible with plugins.
- **PyCharm** – Advanced debugging and environment management.
- **Jupyter Notebook** – Interactive coding for data analysis and visualization.
- **Spyder** – Tailored for scientific and data analysis workflows.

These IDEs provide debugging, syntax highlighting, and visualization tools, enhancing productivity in data science tasks.

---

2. Python Programming Techniques
Python supports several concise and expressive programming techniques that allow efficient manipulation of data. These techniques include lambda functions, higher-order functions (map, filter, reduce), list comprehensions, and generators.



2.1 Lambda Functions
A lambda function is an anonymous, single-line function defined using the `lambda` keyword. It is primarily used for short operations where defining a formal function would be unnecessarily verbose.

Syntax:
```
lambda arguments: expression
```
Example:
```
square = lambda x: x * x
print(square(5))  # Output: 25
```

Use with Higher-Order Functions:
```
nums = [1, 2, 3, 4]
squared = list(map(lambda x: x ** 2, nums))
print(squared)
```
Lambda functions are widely used for quick transformations in data pipelines.

2.2 Map, Filter, and Reduce

These functional programming constructs process iterable data structures in a streamlined manner.

- **Map:** Applies a specified function to every element of an iterable.

```
nums = [1, 2, 3, 4]
result = list(map(lambda x: x * 2, nums))
# Output: [2, 4, 6, 8]
```

- **Filter:** Selects elements from an iterable that satisfy a particular condition.

```
nums = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, nums))
# Output: [2, 4, 6]
```

- **Reduce:** Combines all elements of an iterable into a single cumulative value.

```
from functools import reduce
nums = [1, 2, 3, 4]
total = reduce(lambda a, b: a + b, nums)
    # Output: 10
```

2.3 List Comprehension

List comprehensions provide a concise syntax for creating new lists based on existing sequences.

```
nums = [1, 2, 3, 4]
squares = [x * x for x in nums]
# Output: [1, 4, 9, 16]
```

Generators:

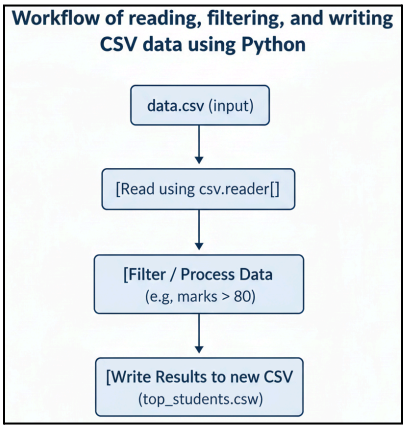
Generators are similar to list comprehensions but produce values lazily (one at a time), thereby conserving memory when dealing with large datasets.

```
gen = (x * x for x in range(5))
for i in gen:
    print(i)
```

---

3. Reading and Manipulating CSV Files

Data scientists frequently work with datasets stored in **Comma-Separated Values (CSV)** format. CSV files represent tabular data in plain text, where each line corresponds to a record and each field is separated by a comma. Python provides both built-in and external libraries to handle CSV files efficiently.



3.1 Using the csv Module

The built-in `csv` module provides control for reading and writing structured text data.

Reading a CSV file:

```
import csv
with open('data.csv', mode='r') as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

Reading with headers:

```
with open('data.csv', mode='r') as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row['Name'], row['Age'])
```

Writing to a CSV file:

```
import csv
with open('output.csv', mode='w', newline='') as file:
    writer = csv.writer(file)
    writer.writerow(['Name', 'Age'])
    writer.writerows([['Alice', 25], ['Bob', 30]])
```

3.2 Using NumPy to Read CSV

For numerical datasets, NumPy provides the function `genfromtxt()` to read data directly into arrays:

Example:

```
import numpy as np
data = np.genfromtxt('data.csv', delimiter=',', skip_header=1)
print(data)
```

3.3 Manipulating Data

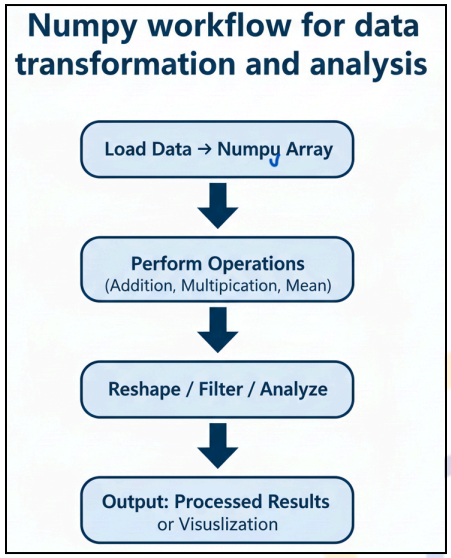
After reading the file, the data can be filtered, modified, or analyzed.

Example: Filtering students who scored more than 80.

```
import csv
rows = []
with open('students.csv', 'r') as file:
    reader = csv.DictReader(file)
    for row in reader:
        if int(row['Marks']) > 80:
            rows.append(row)

with open('top_students.csv', 'w', newline='') as file:
    writer = csv.DictWriter(file, fieldnames=['Name', 'Marks'])
    writer.writeheader()
    writer.writerows(rows)
```

4. NumPy Library

NumPy (Numerical Python) forms the foundation of scientific computing in Python. It provides high-performance, multidimensional array objects and tools for performing complex mathematical operations efficiently.
Unlike Python lists, NumPy arrays are homogeneous and allow vectorized operations, making computation significantly faster.



4.1 Creating Arrays

```
import numpy as np
a = np.array([1, 2, 3, 4])
b = np.array([[1, 2, 3], [4, 5, 6]])
```

4.2 Array Properties

```
print(a.ndim)  # Number of dimensions
print(a.shape)  # Size of each dimension
print(a.size)  # Total number of elements
print(a.dtype)  # Data type of array elements
```

4.3 Special Arrays

NumPy provides built-in methods for creating arrays with specific initial values.

```
np.zeros((2, 3))    # Array of zeros
np.ones((3, 3))     # Array of ones
np.full((2, 2), 7)  # Array filled with 7
np.eye(3)           # Identity matrix
np.arange(0, 10, 2)  # Even numbers from 0 to 8
```

4.4 Array Operations

NumPy allows element-wise operations on arrays.

```
x = np.array([1, 2, 3])
y = np.array([4, 5, 6])

print(x + y)  # Addition
```

```
print(x * y)  # Multiplication
print(x ** 2) # Power
```

4.5 Aggregate Functions

NumPy provides functions to compute statistical and mathematical summaries.

```
a = np.array([1, 2, 3, 4, 5])
print(np.sum(a))   # Sum = 15
print(np.mean(a))  # Mean = 3.0
print(np.max(a))   # Maximum = 5
print(np.min(a))   # Minimum = 1
```

4.6 Reshaping and Transposing

```
a = np.arange(6)
b = a.reshape((2, 3))
print(b)
print(b.T)  # Transpose
```

4.7 Filtering using Conditions

```
a = np.array([10, 20, 30, 40, 50])
print(a[a > 25])  # Output: [30, 40, 50]
```

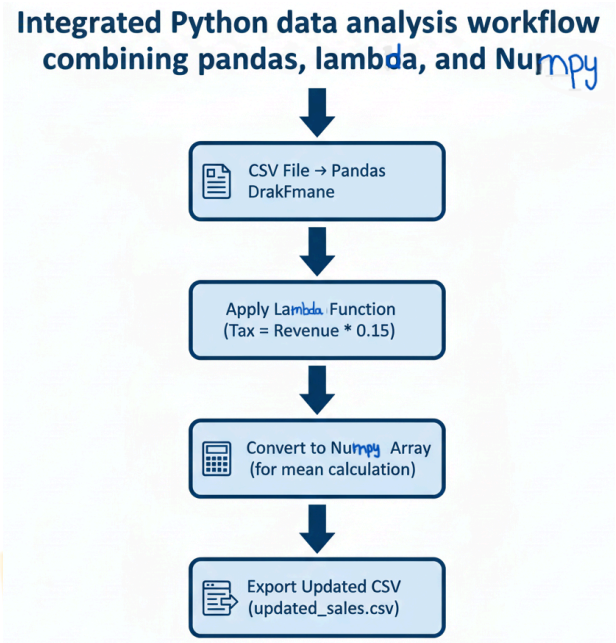
Example: CSV Integration

```
import pandas as pd
import numpy as np

df = pd.read_csv('sales.csv')
sales = df['Revenue'].to_numpy()
mean_sales = np.mean(sales)
print("Average Revenue:", mean_sales)

df['Tax'] = df['Revenue'].apply(lambda x: x * 0.15)
df.to_csv('updated_sales.csv', index=False)
```

This example integrates pandas (CSV handling), lambda (computation), and NumPy (statistics) in one pipeline — representing a typical data analysis workflow.

Integration of Concepts: CSV, Lambda, and NumPy

This diagram summarizes how Python tools combine in a single data analysis workflow.



Summary

Concept	Description	Example / Tool
Python Environment	Setup using IDEs, virtual environments	`venv`, `VS Code`, `Jupyter`
Lambda Functions	Compact one-line functions	`lambda x: x*2`
Map/Filter/Reduce	Functional programming tools	`map(lambda x: x*2, data)`
Reading CSV	Structured file reading	`csv`, `pandas`
NumPy Arrays	Efficient numerical computing	`np.array`, `np.mean()`

Python provides a robust and flexible foundation for data science, combining readability with extensive computational capabilities. Through mastery of the Python environment, programming constructs such as lambda functions and comprehensions, techniques for handling CSV files, and the efficient numerical processing features of NumPy, one develops the essential skill set required for performing advanced data analysis and scientific computation.